

Linearized ADMM for the Schrödinger Bridge on a Dyadically Graded Non-Uniform Time Grid

Companion note to *Computational Methods for the Schrödinger Bridge Problem*

Abstract

This note extends the fully-discrete (Crank–Nicolson-style) Fokker–Planck projection of the main paper’s Appendix A from a uniform time step Δt to an arbitrary, in particular *dyadically graded*, non-uniform time partition $\{\Delta t_j\}_{j=1}^{n_t}$. We show that the entire LADMM recursion (Eq. (35) of the main paper) is unchanged in form – only the definition of the temporal derivative operator D_t changes – and that the per-spatial-mode reduction of the projection step still factors as a sum of a Gram matrix and a nonnegative diagonal term, so the symmetric positive-definiteness result of Lemma A.2 survives *verbatim*, with an identical proof, for *any* admissible $\{\Delta t_j\}$. What does not survive is the closed-form diagonalization used for the singular DC (spatial $k=1$) mode: on a non-uniform grid the discrete time-Laplacian K_t is no longer a constant-coefficient (Toeplitz) operator and is no longer diagonalized by the fixed DCT-II basis T_t , forcing that one mode’s solve from an $O(n_t \log n_t)$ spectral divide down to an $O(n_t^2)$ dense pseudo-inverse application – identified here as the single place efficiency is lost in exchange for the grid’s local accuracy gain. Section 9 carries the same generalization through for the main paper’s ETD scheme (fully closed-form, no probing needed there either, sharing the same DC-mode fallback), though a head-to-head test did not clearly reproduce the stability advantage over CN that motivated doing so. Section 3 shows the same generalization covers a second, structurally different family – a Chebyshev-Gauss-Lobatto-clustered grid – with no new derivation needed, and Section 4 answers a natural follow-up: whether Clenshaw–Curtis quadrature weights should replace the local cell width in the L^2 norms once the grid clusters at Chebyshev points (no – the field is piecewise-constant per cell, not a global polynomial sampled at the nodes, so the composite-midpoint weighting already in use is the consistent choice regardless of clustering pattern). Section 10 derives the complementary fact for the KE prox itself: unlike the constraint-side operators of Sections 5–9, the KE prox’s quadratic penalty was never $\Delta t \Delta x$ -weighted even on the uniform grid (a harmless omission there, since a single shared constant factor never changes an argmin) – so generalizing it correctly means rescaling the *objective* term by each cell’s width relative to the reference grid, $\Delta t_n / \Delta t_{\text{ref}}$, not reusing the diagnostic norms’ $\Delta t_n \Delta x$ weighting. All formulas below were checked against the reference implementation to machine precision (Sections 9 and 12, footnoted).

1 Setup and notation (recap)

We reuse the main paper’s notation throughout. $\Omega = [0, 1]$, $T = 1$, n_x spatial cells, $\Delta x = 1/n_x$. The density/momentum pair (ρ, m) is collocated at cell centers; the staggered pair $(\tilde{\rho}, b)$ carries the hard constraint, with $\tilde{\rho}$ staggered in *time* (fixed endpoint data μ, ν) and b staggered in *space* (homogeneous Neumann/no-flux). D_x, A_x (space derivative/average, $n_x \times (n_x - 1)$) and $K_x = D_x D_x^\top$ are exactly as in the main paper and are untouched by everything below – only the time grid is made non-uniform, space stays uniform throughout.

On a *uniform* time grid (n_t cells, $\Delta t = 1/n_t$), the main paper's Appendix A discretizes the Fokker–Planck constraint as

$$\mathcal{C}_h^\varepsilon = \{(\tilde{\rho}, b) : [D_t \otimes I_{n_x} - \varepsilon(A_t \otimes K_x)] \tilde{\rho} + (I_{n_t} \otimes D_x) b = d\}, \quad (1)$$

and projects onto it via $\text{Proj}_{\mathcal{C}_h^\varepsilon}(\tilde{\rho}, b) = (\tilde{\rho}, b) - \mathcal{L}_\varepsilon^\top (\mathcal{L}_\varepsilon \mathcal{L}_\varepsilon^\top)^+ (\mathcal{L}_\varepsilon(\tilde{\rho}, b) - \frac{1}{\Delta t}(e_1 \otimes \mu - e_{n_t} \otimes \nu))$, which after a spatial DCT reduces (Eq. (38)–(39)) to n_x independent $n_t \times n_t$ tridiagonal solves $M^k w^k = \hat{r}^k$, one per spatial DCT mode $k = 1, \dots, n_x$, with $M^1 = K_t$ singular (handled by a second, temporal, DCT) and M^k ($k > 1$) SPD (Lemma A.2), inverted by an unpivoted Thomas sweep. This whole apparatus – D_t , A_t , M^k , the Thomas solve – is what this note generalizes.

2 The dyadically graded time grid

Fix a base resolution $\bar{n}_t$ and a grading depth $n_b \geq 1$ with $2n_b \leq \bar{n}_t$. Base cell i counted from either boundary ($i = 1$ touches $t = 0$ or $t = 1$; $i = n_b$ is the last graded cell, bordering the untouched interior) is split into 2^{n_b-i+1} equal sub-cells. Writing $h = 1/\bar{n}_t$ for the base width, this gives the non-uniform partition

$$\Delta t_j = \begin{cases} h/2^{n_b-i+1}, & \text{cell } j \text{ is the } i\text{-th graded cell from the nearer boundary, } i = 1, \dots, n_b, \\ h, & \text{cell } j \text{ is in the untouched interior.} \end{cases} \quad (2)$$

The total cell count is $n_t = \bar{n}_t + 2(2^{n_b+1} - 2 - n_b)$ (geometric sum on each end). $n_b = 1$ reproduces the single-cell-split grid exactly ($h/2$, $h/2$ at each end); larger n_b grades $h/2, h/4, \dots, h/2^{n_b}$ inward from the boundary, each cell exactly half the width of its neighbor *one step further from the boundary* – including the seam with the untouched interior, which plays the role of an implicit, unrefined (n_b+1) -th member of the chain, so no special-casing is needed there. (Implementation: `graded_ends_time_grid` in `time_grid.py`; `n_boundary=1` is checked to reduce to the single-cell split exactly, see `validate_nu.py`.)

Let $t_0 = 0 < t_1 < \dots < t_{n_t} = 1$ be the resulting edges and $\tau_j = \frac{1}{2}(t_{j-1} + t_j)$ the cell centers, $j = 1, \dots, n_t$ (1-indexed, matching the main paper's convention; this is a shift from the 0-indexed `t_edges/t_centers` used in the codebase). $\tilde{\rho}$ still lives at the $n_t - 1$ interior edges $t_1, \dots, t_{n_t-1}$ (plus fixed μ, ν at t_0, t_{n_t}); (ρ, m, b) still live at the n_t cell centers τ_j – the grid convention of Section 1 is untouched, only the *spacing* changes.

3 The Chebyshev-clustered time grid

A second, structurally different way to concentrate resolution near the boundaries: instead of grading a fixed number of cells geometrically (Section 2), smoothly reshape *every* cell of the uniform reference grid by a fixed map. Let $\xi_j = j/n_t$, $j = 0, \dots, n_t$, be the uniform reference partition and

$$c(\xi) = \frac{1}{2}(1 - \cos(\pi\xi)), \quad (3)$$

the same map underlying Chebyshev–Gauss–Lobatto (CGL) nodes: on $[-1, 1]$ the n_t+1 CGL nodes are $\cos(\pi j/n_t)$, and c is exactly that same node set, affinely rescaled onto $[0, 1]$ with the endpoint order reversed ($c(0) = 0$, $c(1) = 1$). A strength $s \in [0, 1]$ blends the identity and the full map,

$$t_j = (1 - s)\xi_j + s c(\xi_j), \quad t_0 := 0, \quad t_{n_t} := 1 \text{ pinned exactly}, \quad (4)$$

so $s = 0$ reproduces the uniform grid exactly and $s = 1$ places every edge at a CGL node. (Implementation: `clustered_time_grid` in `time_grid.py`.)

Cell widths: quadratic clustering, not geometric

Unlike Section 2’s dyadic grading – a fixed number of cells, each exactly half its neighbor, with the untouched interior held at a single fixed width $h - c$ reshapes every cell continuously, by an amount that varies smoothly with position; there is no untouched, exactly-uniform interior here. Since $c'(\xi) = \frac{\pi}{2} \sin(\pi\xi)$ vanishes at both endpoints, cell widths near the boundary shrink *quadratically* in n_t , not geometrically: at $s = 1$,

$$\Delta t_j = c(\xi_j) - c(\xi_{j-1}) \approx c'(\xi_{j-1})/n_t = \frac{\pi}{2n_t} \sin(\pi\xi_{j-1}) \xrightarrow{j \rightarrow 0} \frac{\pi^2}{2n_t^2} j, \quad (5)$$

using $\sin(\pi\xi) \approx \pi\xi$ for small ξ – the classical $O(n_t^{-2})$ endpoint spacing of Chebyshev nodes, versus $O(n_t^{-1})$ everywhere on the uniform grid and the fixed, geometric floor $h/2^{n_b}$ of the dyadic grid (Section 2). Mid-domain ($\xi \approx \frac{1}{2}$), $\Delta t_j \approx \pi/(2n_t)$, the same order as the uniform grid.

j (edge)	0	1	2	3	4	5	6	7	8
t_j	0	0.0381	0.1464	0.3087	0.5	0.6913	0.8536	0.9619	1

i (cell)	0	1	2	3	4	5	6	7
Δt_i	0.0381	0.1084	0.1622	0.1913	0.1913	0.1622	0.1084	0.0381

Worked example, $n_t = 8$, $s = 1$ (`clustered_time_grid(8, strength=1.0)`, matches to 10^{-6}). Note $\sum_i \Delta t_i = 1$ and the symmetric, strictly-graded-toward-both-ends profile – every cell distinct, unlike the flat interior plateau of Section 2’s worked example.

Nothing in Sections 5–9 is specific to *how* $\{\Delta t_j\}$ was generated: $D_t(\Delta t)$ (Eq. (7)), the Gram-matrix decomposition (Proposition 2), the SPD bound (Theorem 1), the singular-mode analysis (Proposition 3), and the ETD generalization (Section 9) were all proved for *any* admissible partition – so this Chebyshev-clustered grid needs no new derivation, only a numeric check that it actually satisfies the hypotheses those proofs assumed (positive, well-ordered Δt_j). `validate_nu.py`’s adjoint-identity and repeated-projection-stability checks already include a `clustered` case; a fresh end-to-end run (`clustered_time_grid(64, strength=0.8)`, $\varepsilon = 10^{-3}$, CN scheme) converges and matches the analytical SB solution to mean L^2 error 1.0×10^{-2} , no special-casing required anywhere.

4 Quadrature: composite midpoint vs. Clenshaw–Curtis

Since Section 3’s edges cluster at Chebyshev(-Gauss-Lobatto) points, a natural question is whether integrals over time – the L^2 norms driving the LADMM stopping criteria, or the kinetic-energy functional J_h itself – should use Clenshaw–Curtis (CC) quadrature weights, the weights classically paired with Chebyshev nodes to achieve near-spectral accuracy, in place of the local cell width Δt_j already used throughout.

No. CC weights are derived by integrating the unique global polynomial (equivalently, truncated Chebyshev series) that interpolates function values *at* the Chebyshev nodes – the

correct quadrature only when that is how the integrand is represented. Ours is not: (ρ, m, b) are a piecewise-constant-per-cell (finite-volume) field, exactly as on the uniform or dyadically graded grid, living at the cell *centers* τ_j – the midpoints between consecutive edges, a set distinct from the edges t_j themselves (where the CGL nodes actually sit). CC quadrature has no direct meaning for a field sampled at this dual, staggered point set.

The quadrature consistent with a piecewise-constant field is the one already implemented, grid-generically, in `pipeline_nu.py`'s `_weighted_sq_sum`:

$$\int_0^1 f(t)^2 dt \approx \sum_j f(\tau_j)^2 \Delta t_j \quad (6)$$

(composite midpoint rule – with the center-to-center dual width $\Delta t_j^{\text{dual}} = \frac{1}{2}(\Delta t_j + \Delta t_{j+1})$ used instead for the staggered $\tilde{\rho}$, which lives at edges rather than centers). This is $O(\Delta t_j^2)$ locally accurate for *any* smoothly-varying partition – uniform, dyadically graded, or Chebyshev-clustered alike – so nothing grid-specific was needed: the same formula already covers this section's grid unchanged.

Remark 1 (Why upgrading the quadrature alone would not help). *Even a genuine CC-style upgrade would not raise the overall solution accuracy: the scheme's error is bottlenecked by the $O(\Delta t^2)$ -ish local truncation error of $D_t(\Delta t)$ (Section 6.1), a low-order finite-difference stencil, not by the quadrature used to measure norms after the fact. Matching a high-order quadrature to a low-order discretization buys nothing – the weaker of the two still dominates the error. A genuinely higher-order scheme would require replacing D_t itself with a Chebyshev differentiation matrix (dense, global, collocated directly at the CGL nodes) – a structurally different method giving up the staggered grid and the DCT-diagonalization / local Thomas-solve architecture this whole note relies on. A legitimate but separate undertaking, not attempted here.*

Remark 2 (A related, smaller issue – diagnostics, not the solver). *The comparison script's printed summary statistics (`err_cc.mean()`, `.max()`) are a plain, unweighted **numpy** mean/max over the n_t rows, not Δt_j -weighted. On a strongly clustered grid, with far more rows packed near the boundary, this over-represents the boundary region relative to a genuine time-averaged error. Unlike the norms above (used inside the LADMM iteration itself, already correctly weighted), this is only a diagnostic print in `scripts/sb_gaussian_refine_ends_vs_uniform.py` and does not affect the solve – flagged here for completeness, not fixed.*

5 Generalized operators

5.1 The time derivative $D_t(\Delta t)$

D_t becomes row-dependent: writing $\Delta t_j = t_j - t_{j-1}$,

$$D_t(\Delta t) = \begin{pmatrix} 1/\Delta t_1 & & & & \\ -1/\Delta t_2 & 1/\Delta t_2 & & & \\ & \ddots & \ddots & & \\ & & -1/\Delta t_{n_t-1} & 1/\Delta t_{n_t-1} & \\ & & & -1/\Delta t_{n_t} & \end{pmatrix} \in \mathbb{R}^{n_t \times (n_t-1)}, \quad (7)$$

i.e. row j (mapping the two edges bounding cell j to that cell's centered difference) uses *only* Δt_j – this is a per-row rescaling of the uniform-grid D_t , nothing more, since each row only ever

touches its own cell's two bounding edges. Reduces to the main paper's D_t exactly when every $\Delta t_j = \Delta t$.

5.2 The time average A_t is unchanged

Lemma 1 (Midpoint averaging is grid-independent). *A_t – the plain $\frac{1}{2}, \frac{1}{2}$ averaging matrix of Eq. (15) – needs no modification on a non-uniform grid: it remains the exact linear interpolant of the two bounding edge values at the cell center, for any Δt_j .*

Proof. Let $L(t) = v_{j-1} + (v_j - v_{j-1}) \frac{t-t_{j-1}}{\Delta t_j}$ be the affine interpolant of the two edge values bounding cell j . At the cell center $\tau_j = t_{j-1} + \Delta t_j/2$, $L(\tau_j) = v_{j-1} + \frac{1}{2}(v_j - v_{j-1}) = \frac{1}{2}(v_{j-1} + v_j)$, independent of Δt_j . $\square$

Consequently A_t , K_x , D_x , A_x are all completely unaffected by grading the time grid; *only* D_t changes.

5.3 The discrete constraint set

Substituting $D_t(\Delta t)$ for D_t in (1) gives the graded-grid analogue

$$\begin{aligned} \mathcal{C}_h^\varepsilon(\Delta t) = \{(\tilde{\rho}, b) : [D_t(\Delta t) \otimes I_{n_x} - \varepsilon(A_t \otimes K_x)] \tilde{\rho} + (I_{n_t} \otimes D_x) b = d\}, \\ d = \frac{1}{\Delta t_\bullet} (e_1 \otimes \mu - e_{n_t} \otimes \nu) \Big|_{\text{row-scaled}}, \end{aligned} \quad (8)$$

where the boundary rows of d inherit their own $\Delta t_1 / \Delta t_{n_t}$ scaling from $D_t(\Delta t)$'s first/last row (the interior of d is zero, exactly as before). **Remark A.1**'s solvability condition ($\sum_i \mu_{i+1/2} = \sum_i \nu_{i+1/2}$) is unchanged: it only used $1_{n_x}^\top K_x = 0$, a purely spatial fact.

Proposition 1 (LADMM is grid-agnostic). *The LADMM recursion (35a)–(35c) of the main paper is unchanged in form on the graded grid: only the definition of $\mathcal{L}_\varepsilon$ (through $D_t(\Delta t)$) changes. Prox_{J_h} (Proposition 3.2) is entirely pointwise in (ρ, m) and never references D_t , so it is untouched; the consensus map $\mathcal{A}$ interpolates $\tilde{\rho}$ only in time and b only in space (Eq. (16)), and by Lemma 1 its time half (A_t) also needs no change.*

6 The projection step

Write $D_t := D_t(\Delta t)$ for brevity. Following the same DCT-in-space block-diagonalization as the main paper (space is still uniform, so this step is verbatim unchanged), the projection reduces to n_x independent $n_t \times n_t$ systems $M^k w^k = \hat{r}^k$, $k = 1, \dots, n_x$, one per spatial DCT mode with eigenvalue λ_x^k .

Proposition 2 (Gram-matrix decomposition, any grid).

$$M^k = M_0 + \lambda_x^k M_1 + \varepsilon^2 (\lambda_x^k)^2 M_2, \quad M_0 = D_t D_t^\top, \quad M_1 = \varepsilon (D_t A_t^\top + A_t D_t^\top) + I_{n_t}, \quad M_2 = A_t A_t^\top, \quad (9)$$

and equivalently

$$M^k = (D_t + \varepsilon \lambda_x^k A_t) (D_t + \varepsilon \lambda_x^k A_t)^\top + \lambda_x^k I_{n_t}. \quad (10)$$

Proof. Expand (10): $(D_t + \varepsilon \lambda_x^k A_t)(D_t + \varepsilon \lambda_x^k A_t)^\top = D_t D_t^\top + \varepsilon \lambda_x^k (D_t A_t^\top + A_t D_t^\top) + \varepsilon^2 (\lambda_x^k)^2 A_t A_t^\top$. Adding $\lambda_x^k I_{n_t}$ and regrouping by powers of λ_x^k gives exactly (9). Neither step used any structural property of D_t (Toeplitz-ness, constant coefficients) – it is pure algebra, valid for the matrix (7) exactly as it is for the uniform-grid D_t .¹ $\square$

This is *identical in shape* to the main paper’s Eq. (39) – the closed-form entries α^j, β^j are gone (replaced by row-dependent $1/\Delta t_j, 1/\Delta t_{j+1}$ terms, see Section 6.1), but the algebraic structure $M^k = (\text{Gram}) + \lambda_x^k I$ survives completely.

Theorem 1 (SPD survives verbatim). *For every k with $\lambda_x^k > 0$, M^k is symmetric positive definite with $\lambda_{\min}(M^k) \geq \lambda_x^k$ – the exact bound of Lemma A.2, unconditional in $\{\Delta t_j\}$.*

Proof. Symmetry is immediate from (10) (a Gram matrix plus a multiple of the identity). For any unit v , $\langle v, M^k v \rangle = \|(D_t + \varepsilon \lambda_x^k A_t)^\top v\|^2 + \lambda_x^k \|v\|^2 \geq \lambda_x^k$, using only that $(D_t + \varepsilon \lambda_x^k A_t)(\cdot)^\top$ is a Gram matrix (hence PSD) – verbatim the main paper’s Lemma A.2 proof, which likewise never used the Toeplitz structure of D_t . $\square$

Because M^k is SPD, the unpivoted Thomas sweep used for $k > 1$ is backward stable for the same classical reason it is on the uniform grid (symmetric Gaussian elimination on an SPD tridiagonal matrix is unconditionally stable, independent of the M-matrix argument the main paper also invokes) – *this generalizes for free*. The generalized Thomas sweep itself (`thomas_batch_precomp_general` / `thomas_batch_solve_general` in `projection_nu.py`) only needs a row-varying off-diagonal, which is exactly what (9)’s M_0, M_1 produce; it is still $O(n_t)$ work, vectorized across all $n_x - 1$ non-DC modes simultaneously via array broadcasting – *identical* asymptotic cost to the uniform-grid batched solve.

6.1 Row entries of M_0, M_1

Unlike the uniform grid, $M_0 = D_t D_t^\top$ is not Toeplitz: its interior rows have $(M_0)_{jj} = 1/\Delta t_j^2 + 1/\Delta t_{j+1}^2$, $(M_0)_{j,j+1} = (M_0)_{j+1,j} = -1/\Delta t_{j+1}^2$, with the familiar boundary rows adjusted for the missing neighbor. Concretely, the adjoint entering M_0 (and, via (9), every M^k) is

$$(D_t^\top v)_j = \frac{v_{j+1}}{\Delta t_{j+1}} - \frac{v_j}{\Delta t_j}, \quad (11)$$

so at a fine/coarse junction (e.g. $\Delta t_j = h/2$, $\Delta t_{j+1} = h$) the two terms are asymmetrically weighted by a factor of 2 – each term rescaled by its *own* cell’s width, not by any shared center-to-center distance (see `docs/nonuniform_grid_refine_ends.tex` for a fully worked $\bar{n}_t = 6$ numeric instance of this row).

7 The singular DC mode

M^1 (the $k=1$, $\lambda_x^k = 0$ mode) equals $M_0 = D_t D_t^\top$, which is rank-deficient by construction: $D_t \in \mathbb{R}^{n_t \times (n_t - 1)}$, so $D_t^\top \in \mathbb{R}^{(n_t - 1) \times n_t}$ has rank at most $n_t - 1 < n_t$, hence $M_0 = D_t D_t^\top$ is always singular – a shape argument, true on any grid.

¹Numerically checked against the codebase’s own construction of M_0, M_1, M_2 (built by probing `deriv_t_at_rho/interp_t_at_rho` with unit vectors, `projection_nu.py`) on a genuinely graded grid: all three matrices, and the factored identity (10), match to 10^{-8} – 10^{-10} .

Proposition 3 (The null direction is Δt itself, not 1_{n_t}). $\ker M_0 = \ker D_t^\top = \text{span}\{\Delta t\}$, where $\Delta t = (\Delta t_1, \dots, \Delta t_{n_t})^\top$ is the vector of cell widths – not $\text{span}\{1_{n_t}\}$ unless the grid is uniform.

Proof. $D_t^\top v = 0$ means $v_{j+1}/\Delta t_{j+1} = v_j/\Delta t_j$ for every j , i.e. $v_j/\Delta t_j$ is constant in j , i.e. $v_j = C\Delta t_j$. On a uniform grid $\Delta t_j \equiv \Delta t$, so this reduces to $v = \text{const}$, recovering the main paper’s fact that $K_t 1_{n_t} = 0$.² $\square$

Remark 3 (A gauge mismatch with the continuum). *In the continuous problem the null direction of the (unregularized) time-Laplacian on ϕ is the constant function $\phi \equiv c$ (adding a constant to the dual potential is the SB problem’s genuine gauge freedom). Proposition 3 shows the discrete null direction drifts away from 1_{n_t} toward Δt once the grid is graded. This is a symptom of using the Euclidean (array-index) adjoint for $D_t^\top$ rather than a Δt -weighted- L^2 adjoint that would keep the continuum’s constant-function gauge direction exactly. The weighted adjoint is the more “physically natural” choice, but (verified empirically in this codebase – see `operators.nu.py`’s docstring) it is not the correct adjoint for the LADMM projection identity to hold: substituting it silently breaks the algebra behind (10) and the projection stops being idempotent under repeated application. So there is a genuine, currently unresolved tension between “discrete gauge direction matches the continuum” and “the projection is a provably well-posed orthogonal projection” – this note does not resolve it, only documents it as an open point.*

7.1 Loss of closed-form diagonalization

On the uniform grid, $K_t = D_t D_t^\top$ is exactly the constant-coefficient discrete Neumann Laplacian, simultaneously diagonalized by the fixed orthogonal DCT-II matrix T_t (this is why Algorithm 2, line 3, can solve the $k=1$ mode in $O(n_t \log n_t)$: one forward DCT, one division by the n_t known eigenvalues λ_t^i , one inverse DCT). Once Δt_j varies, $K_t(\Delta t) = D_t(\Delta t) D_t(\Delta t)^\top$ is *not* a constant-coefficient operator and, in general, is not simultaneously diagonalized by *any* fixed fast-transform basis – T_t in particular no longer diagonalizes it (it only diagonalizes the specific Toeplitz structure the uniform grid produces). The reference implementation falls back to a dense pseudo-inverse, `M0_pinv = np.linalg.pinv(M0)`, computed once at precompute time and applied as a dense $n_t \times n_t$ matrix–vector product at *every* LADMM iteration (`projection.nu.py`: `phi_hat[:, 0] = bp.M0_pinv @ f_hat[:, 0]`). This is exactly, and only, where efficiency is lost – quantified in Section 12.

8 Algorithm

9 Generalizing the ETD scheme

Section 3.2 of the main paper’s ETD scheme integrates the diffusion term exactly per DCT- x mode: for mode k with rate $\alpha_j = \varepsilon \lambda_x^k \Delta t$, $c_j = e^{-\alpha_j}$, $\varphi_j = (1 - c_j)/\alpha_j$, the constraint becomes the exact-semigroup recursion $(\hat{q}_h^{j+1} - c_j \hat{q}_h^j)/\Delta t + \varphi_j (\widehat{D_x b_h})^j = 0$ (Eq. (26)). Unlike the CN scheme’s D_t (Section 5), this recursion’s coefficients c_j, φ_j themselves depend on Δt through the exponential – so generalizing to $\{\Delta t_n\}$ means letting *every cell* integrate the exact semigroup

²Verified numerically: on a genuinely graded grid, $\|M_0 \Delta t\|/\|\Delta t\| \sim 10^{-13}$ while $\|M_0 1_{n_t}\|/\|1_{n_t}\| = O(10^3)$ – the constant vector is emphatically *not* in the kernel once the grid is non-uniform.

Algorithm 1 Precompute for $\text{Proj}_{C_h^\varepsilon}(\Delta t)$, graded grid

Require: $\{\Delta t_j\}_{j=1}^{n_t}, \varepsilon$

- 1: Probe $D_t(\Delta t)$, A_t against n_t-1 unit vectors each to assemble M_0, M_1, M_2 $\triangleright O(n_t^2)$ – was $O(n_t)$ closed-form, Eq. (39)
 - 2: Extract tridiagonal entries of M_0, M_1, M_2 ; form $\{\lambda_x^k\}_{k=2}^{n_x}$ -batched diagonal/off-diagonal via (9) $\triangleright O(n_t n_x)$, unchanged
 - 3: Thomas forward sweep (batched over $k = 2, \dots, n_x$) $\triangleright O(n_t n_x)$, unchanged
 - 4: $M_0^+ \leftarrow$ dense pseudo-inverse of M_0 $\triangleright O(n_t^3)$ – was $O(n_t)$ (eigenvalues only), Algorithm 2 line 3
-

Algorithm 2 $\text{Proj}_{C_h^\varepsilon}(\tilde{\rho}, b)$, per LADMM iteration

Require: $(\tilde{\rho}, b)$, precomputed M_0^+ , Thomas factors

- 1: $r \leftarrow \mathcal{L}_\varepsilon(\tilde{\rho}, b) - d$; spatial DCT of r $\triangleright O(n_t n_x \log n_x)$, unchanged
 - 2: $k = 1$: $\hat{w}^1 \leftarrow M_0^+ \hat{r}^1$ $\triangleright O(n_t^2)$ – **was** $O(n_t \log n_t)$, Algorithm 2 line 3
 - 3: $k = 2, \dots, n_x$: batched Thomas back-substitution $\triangleright O(n_t n_x)$, unchanged
 - 4: Inverse spatial DCT; apply $\mathcal{L}_\varepsilon^\top$ $\triangleright O(n_t n_x \log n_x)$, unchanged
-

over *its own* width, exactly the same "cell-local" principle behind $D_t(\Delta t)$ (Section 5.1):

$$c_n(\lambda_x^k) = e^{-\varepsilon \lambda_x^k \Delta t_n}, \quad \varphi_n(\lambda_x^k) = \frac{1 - c_n(\lambda_x^k)}{\varepsilon \lambda_x^k \Delta t_n} \quad (\rightarrow 1 \text{ as } \varepsilon \lambda_x^k \Delta t_n \rightarrow 0), \quad (12)$$

promoting the uniform grid's per-mode scalars to $(n_t \times n_x)$ arrays, one value per (cell, spatial mode) pair – cheap, $O(n_t n_x)$, an outer product $\varepsilon \Delta t \otimes \lambda_x$ through `exp`. The residual for cell n is the bidiagonal recursion $R^k(w)[n] = (w[n+1] - c_n(\lambda_x^k)w[n])/\Delta t_n$, and (unlike the CN case) *every* downstream formula stays closed-form – no numerical probing is needed anywhere in this generalization, because R^k is already an explicit two-term recursion, not a composition of several operators:

$$T^k = R^k R^{k\top} + \text{diag}(\varphi_n(\lambda_x^k)^2) \lambda_x^k, \quad (13)$$

$$(R^k R^{k\top})_{nn} = \frac{c_n(\lambda_x^k)^2}{\Delta t_n^2} [n \geq 1] + \frac{1}{\Delta t_n^2} [n \leq n_t - 2], \quad (R^k R^{k\top})_{n,n+1} = -\frac{c_{n+1}(\lambda_x^k)}{\Delta t_n \Delta t_{n+1}}.$$

The diagonal-only $\varphi_n(\lambda_x^k)^2 \lambda_x^k$ term (rather than a tridiagonal contribution, cf. M_1, M_2 in (9)) reflects that $\varphi_n(\lambda_x^k)$ only ever rescales cell n 's *own* flux term, and D_x, A_x never mix time rows – so the momentum-correction's feedback into T^k is exactly diagonal, with no cross-row coupling to derive. Both (13) lines were checked to reduce to `precomp_expsemi_proj`'s known-good closed-form D, e arrays (main paper Algorithm 1) to machine precision on a uniform grid, and the correction formulas (ρ : $\hat{\delta}\rho[n] = c_{n+1}(\lambda_x^k)\hat{w}[n+1]/\Delta t_{n+1} - \hat{w}[n]/\Delta t_n$; m : unchanged in form, $\varphi_n(\lambda_x^k)$ now row-dependent) were likewise checked end-to-end against `proj_fokker_planck_expsemi` to 10^{-14} (`validate_nu.py`, checks 6–8). The DC mode ($k=1, \lambda_x^k=0$) again has $c_n \equiv 1$ for every n , so $T^1 = R^1 R^{1\top} = D_t(\Delta t) D_t(\Delta t)^\top$ *exactly* – literally the same singular system as the CN scheme's M_0 (Section 7), handled identically (dense pseudo-inverse; Proposition 3's null vector applies unchanged). Implementation: `projection_expsemi_nu.py`.

Remark 4 (An empirical surprise). *The main paper's Fig. 6 motivation for preferring ETD is CN developing oscillatory ringing once $\varepsilon/\Delta t \gtrsim 1$. Grading pushes exactly there locally, so*

ETD-nu was expected to show a visible edge over CN-nu on a graded grid at moderate-large ε . Testing head-to-head ($\bar{n}_t = 64$, $n_b = 5$ – smallest cell $\Delta t_{\min} \approx 4.9 \times 10^{-4}$, so $\varepsilon/\Delta t_{\min}$ ranges from ~ 20 to ~ 2000 over $\varepsilon \in \{0.1, 1, 4\}$): both schemes converged to comparable residuals and produced comparably small, comparable-magnitude negative excursions in $\tilde{\rho}$ (order 10^{-4} – 10^{-3} , both schemes, all three ε) – no dramatic CN instability materialized in this test. This is consistent with Theorem 1: M^k ’s SPD bound holds unconditionally in $\{\Delta t_j\}$ and ε , so whatever caused the ringing in the main paper’s Fig. 6 test evidently needed more than $\varepsilon/\Delta t \gtrsim 1$ locally at a few cells to manifest (a globally under-resolved uniform grid, per Fig. 6’s own caption, is a different regime than a few highly-refined graded cells embedded in an otherwise well-resolved grid). This note does not resolve the discrepancy – it is left as an open empirical question rather than the confirmed win the theory predicted going in.

10 The kinetic-energy prox

The other half of the LADMM splitting – the y -update, Prox_{J_h} (Proposition 3.2 of the main paper) – needs re-examining for a reason distinct from anything in Sections 5–9: those were all about the *constraint* ($\mathcal{L}_\varepsilon$, the hard equality enforced by projection); this is about the *objective*, and about how the LADMM iteration’s internal penalty relates to the genuine, Δt -weighted physical L^2 norm used everywhere else (Section 4).

Notation: the pointwise density J vs. the array-level objective

The kinetic-energy density is exactly the main paper’s Eq. (3)/(6),

$$J(\rho, m) = \begin{cases} m^2/(2\rho), & \rho > 0, \\ 0, & (\rho, m) = (0, 0), \\ +\infty, & \text{otherwise,} \end{cases} \quad (14)$$

a single closed, proper, convex function of a *scalar pair* $(\rho, m) \in \mathbb{R} \times \mathbb{R}$ (the perspective function of $m^2/2$). The y -update solves, for the *whole* $(n_t \times n_x)$ array (ρ, m) at once,

$$(\rho, m)^{k+1} = \arg \min_{\rho, m} F(\rho, m) + \frac{\gamma}{2} \|(\rho, m) - z\|_2^2, \quad (15)$$

$$\|(\rho, m) - z\|_2^2 := \sum_{n=1}^{n_t} \sum_{i=1}^{n_x} \left[(\rho_{n,i} - \rho_{n,i}^z)^2 + (m_{n,i} - m_{n,i}^z)^2 \right], \quad (16)$$

where $\|\cdot\|_2$ is the *plain, unweighted* (array-index/Frobenius) Euclidean norm over every cell jointly – *not* the $\Delta t_n \Delta x$ -weighted norm `norm_fn` measures (Section 4; see the remark at the end of this section) – and $F(\rho, m) := \sum_{n,i} J(\rho_{n,i}, m_{n,i})$ is the *unweighted* sum of the pointwise density (14). This F , not the physically-normalized $J_h(\rho, m) := \sum_{n,i} \Delta t_n \Delta x J(\rho_{n,i}, m_{n,i})$ that actually approximates $\iint J dx dt$, is what `prox_ke_cc` literally implements.

The uniform-grid convention

Both F and the norm in (15) are sums of independent per-cell terms, so the minimization separates exactly into $n_t n_x$ scalar-pair problems, one per cell:

$$(\rho, m)_{n,i}^{k+1} = \text{Prox}_{\sigma J}(\rho_{n,i}^z, m_{n,i}^z) := \arg \min_{\rho, m} J(\rho, m) + \frac{1}{2\sigma} \left[(\rho - \rho_{n,i}^z)^2 + (m - m_{n,i}^z)^2 \right], \quad \sigma = 1/\gamma, \quad (17)$$

this time with the *ordinary* $\mathbb{R}^2$ Euclidean distance for that single cell – exactly `prox_ke_cc`’s cubic solve, called with `sigma=1/gamma`, no $\Delta t, \Delta x$ anywhere. This is harmless on a uniform grid: since $J_h = \Delta t \Delta x \cdot F$ there ($\Delta t \Delta x$ the same constant factored out of every cell), minimizing F and minimizing J_h have the *same* minimizer; γ, τ were simply tuned, empirically, for whichever global constant multiple of the physical objective F happens to be – costing nothing in correctness, only in how those constants are calibrated for convergence *speed*.

Why this stops being free once Δt varies

That cancellation used one fact: $J_h = \Delta t \Delta x \cdot F$ only for a *single shared* $\Delta t \Delta x$. Once Δt_n varies by row, $J_h(\rho, m) = \sum_{n,i} \Delta t_n \Delta x J(\rho_{n,i}, m_{n,i})$ is no longer F times a global constant – it is F with a genuinely row-dependent reweighting, and computing $\text{Prox}_{(1/\gamma)F}$ (reusing (17) unmodified) would minimize the *wrong* function: a narrow cell’s row would be penalized in the KE term exactly as much as a wide cell’s, even though its true contribution to $\int J dt$ is proportionally smaller.

The fix: rescale σ by the cell’s relative width

The literal physical fix – replace F by J_h throughout – would separate exactly as (17) did, but with $\sigma_n = \Delta t_n \Delta x / \gamma$ instead of $1/\gamma$: correct, but it also rescales the *overall* magnitude of σ by $\Delta t_n \Delta x \sim 1/(n_t n_x)$, forcing γ, τ to be recalibrated from scratch (they were tuned against F , not J_h). `prox_ke_cc_nu` (`prox_nu.py`) instead keeps F ’s calibration and reweights only *relative to the grid’s own average width*, so nothing changes on average: row n solves, *independently for each spatial index i* , the scalar-pair problem

$$(\rho, m)_{n,i}^{k+1} = \arg \min_{\rho, m} \frac{\Delta t_n}{\Delta t_{\text{ref}}} J(\rho, m) + \frac{1}{2\sigma} \left[(\rho - \rho_{n,i}^z)^2 + (m - m_{n,i}^z)^2 \right], \quad \Delta t_{\text{ref}} := \frac{1}{n_t} = \overline{\Delta t_n}, \quad (18)$$

exactly n_x separate cubic solves per row n , all sharing the same weight $\Delta t_n / \Delta t_{\text{ref}}$ (which depends only on n , since Δx is uniform across i) but each seeing its *own* $\rho_{n,i}^z, m_{n,i}^z$ and producing its *own* $(\rho, m)_{n,i}^{k+1}$ – not one vector-valued problem solved jointly across i . Exactly 1 for every row when the grid *is* uniform, recovering (17) verbatim. Multiplying through by σ and absorbing $\Delta t_n / \Delta t_{\text{ref}}$ into an effective step size turns (18) back into the *unmodified* prox (17) with

$$\sigma_{\text{eff}}[n] = \sigma \frac{\Delta t_n}{\Delta t_{\text{ref}}}, \quad (19)$$

exactly `prox_nu.py`’s formula, applied row-by-row to the same cubic solve (17) uses – no new algebra in the cubic, only in what step size each row hands it. Δx needs no analogous correction: it is uniform across x for every row alike, so (exactly as Δt was on the uniform grid) it stays a shared constant already absorbed into however γ was calibrated, never appearing explicitly.

Would a symmetrically-weighted penalty be more consistent?

A tempting alternative to (18): weight the *penalty* by $\Delta t_n \Delta x$ too, so both terms of the y -update are literally Riemann-sum approximations of the same continuous integral – again, one scalar-pair problem per (n, i) , sharing n ’s weight:

$$(\rho, m)_{n,i}^{k+1} = \arg \min_{\rho, m} \Delta t_n \Delta x J(\rho, m) + \frac{\gamma}{2} \Delta t_n \Delta x \left[(\rho - \rho_{n,i}^z)^2 + (m - m_{n,i}^z)^2 \right]. \quad (20)$$

Dividing through by the shared positive constant $\Delta t_n \Delta x$ makes it cancel *completely*, leaving exactly $(\rho, m)_{n,i}^{k+1} = \arg \min_{\rho, m} J(\rho, m) + \frac{\gamma}{2}[(\rho - \rho_{n,i}^z)^2 + (m - m_{n,i}^z)^2]$ – the *same, constant* $\sigma = 1/\gamma$ prox as (17), with *no* row dependence at all. This is not special to this problem: whenever the *same* weight multiplies both the objective term and the penalty term of a separable prox, it cancels out of the argmin identically, regardless of what that weight is. So (18)’s row dependence is only possible *because* it is asymmetric – weighting J but not the penalty – and any ”more consistent”-looking symmetric weighting would silently discard it.

Whether that asymmetry is the right trade is exactly the open question this note flags rather than resolves. The penalty term in (20) is not private to the y -update – it is (half of) the *same* shared augmented-Lagrangian term $\frac{\gamma}{2} \|Ax - y\|^2$ that the x -update (the projection of Sections 5–9) is implicitly extremized against. Weighting it symmetrically only in the y -update, without doing the same in the x -update, would make the two halves of ADMM implicitly extremize *different*-metric approximations of what is supposed to be one shared penalty term. Doing it consistently in *both* halves would require the projection to use the Δt -weighted- L^2 adjoint instead of the Euclidean one – and Remark 3 already documents, empirically, that this specific substitution breaks the projection (it stops being idempotent). So a fully weight-consistent ADMM is not currently available as a drop-in change: (18) is what the reference implementation uses *today*, a deliberate asymmetric choice that produces useful row-dependence precisely because it does *not* try to be a symmetric, ”fully physical” weighting of the whole augmented Lagrangian – not a proof that it is the uniquely correct one.

Remark 5 (Two different norms, two different roles – not to be confused). *This section and Section 4 weight Δt_n into two different places for two different reasons, and they need not agree: `norm_fn/dual_norm_fn` $\Delta t_n \Delta x$ -weight the diagnostic residual norms used only for the LADMM stopping criterion, entirely outside the iterate updates; `prox_ke_cc_nu`’s $\Delta t_n / \Delta t_{\text{ref}}$ reweights the objective inside the y -update itself, changing what point the prox returns. Conflating the two would either double-count the weighting (using $\Delta t_n \Delta x$ instead of the dimensionless $\Delta t_n / \Delta t_{\text{ref}}$ inside the prox) or under-correct it (using the diagnostic norm’s weighting, which also carries a Δx factor the prox never needed).*

This reduces to the uniform-grid pipeline exactly: `validate_nu.py`’s full-pipeline reduction checks (CN and its ETD counterpart alike) already match the *full* LADMM trajectory – prox included – to 10^{-4} against the uniform-grid pipeline; this section derives what those checks were silently already confirming.

11 Explicit norms of every prox, side by side

The LADMM loop (`ladmm.py`) has exactly two proxies – `prox_f1` (the x -update, the FP projection of Sections 5–9) and `solve_y` (the y -update, the KE prox of Section 10) – plus the diagnostic-only `norm_fn/ dual_norm_fn` of Section 4. This section puts the norm each one *actually* uses side by side, then works out explicitly what replacing each with a genuine $\Delta t_n \Delta x$ -weighted (Riemann-sum) norm would look like.

What the code minimizes today

1. ***x*-update / FP projection** (`prox_f1` $\rightarrow$ `proj_fokker_planck_banded_nu / _expsemi_nu`):

$$(\tilde{\rho}, b)^{k+1} = \arg \min_{(\tilde{\rho}, b) \in \mathcal{C}_h^\varepsilon(\Delta t)} \underbrace{\sum_{j,i} (\tilde{\rho}_{j,i} - \mu_{j,i})^2}_{\text{staggered } \tilde{\rho}, n_t-1 \text{ edges}} + \underbrace{\sum_{n,i} (b_{n,i} - \beta_{n,i})^2}_{\text{staggered } b, n_t \text{ centers}}, \quad (21)$$

the *plain, unweighted* array-index Euclidean norm on both variables – no Δt , no Δx , anywhere. This is a hard-constraint (indicator-function) prox: `prox_f1`'s `step` argument is passed through but the projection functions themselves take no step/sigma parameter at all, because the prox of an indicator function is the projection *regardless* of the prox parameter – so the only thing that can vary here is which norm defines "closest," never a magnitude. It is this norm – via the plain transposes $D_t^\top, A_t^\top$ – that Section 5's Gram + $\lambda_x^k I$ structure (Proposition 2) is built from.

2. ***y*-update / KE prox** (`solve_y` $\rightarrow$ `prox_ke_cc_nu`), restating (18):

$$(\rho, m)_{n,i}^{k+1} = \arg \min_{\rho, m} \frac{\Delta t_n}{\Delta t_{\text{ref}}} J(\rho, m) + \underbrace{\frac{\gamma}{2} [(\rho - \rho_{n,i}^z)^2 + (m - m_{n,i}^z)^2]}_{\text{plain, unweighted}}, \quad (22)$$

again a plain Euclidean penalty; only the *objective* J picks up a weight, and even that is the *relative* $\Delta t_n / \Delta t_{\text{ref}}$, not the absolute physical $\Delta t_n \Delta x$ – Δx never appears (harmless, since Δx is the same constant for every entry: see the end of Section 10).

3. **Diagnostics only** (`norm_fn/dual_norm_fn`, Section 4): the genuine $\Delta t_n \Delta x$ Riemann-sum norm, $\Delta x \sum_j \Delta t_j^{\text{dual}} \tilde{\rho}_j^2 + \Delta x \sum_n \Delta t_n m_n^2$ – but this is used *only* to decide when to stop iterating (Section 4's $\varepsilon_D, \varepsilon_P$); it never enters the formula that produces x^{k+1} or y^{k+1} .

Neither prox that actually *defines* an iterate uses the diagnostic norm. This is not an oversight to reconcile casually – the rest of this section shows why fully doing so is either free (the *y*-update: it cancels) or blocked on an open problem (the *x*-update).

y-update: already worked out

Weighting *both* terms of (22) by the same $\Delta t_n \Delta x$ ((20)) cancels identically, collapsing back to the plain, unweighted, constant- $\sigma = 1/\gamma$ prox (17) – no row dependence survives. See the subsection "Would a symmetrically-weighted penalty be more consistent?" above for the full derivation.

x-update: what a $\Delta t_n \Delta x$ -weighted projection would look like

Define diagonal weight matrices for the physical ($\Delta t_n \Delta x$ -Riemann-sum) inner product on each staggered variable – exactly `norm_fn`'s weights (Section 4):

$$W_{\tilde{\rho}} = \Delta x \cdot \text{diag}(\Delta t^{\text{dual}}) \in \mathbb{R}^{(n_t-1) \times (n_t-1)}, \quad W_b = \Delta x \cdot \text{diag}(\Delta t) \in \mathbb{R}^{n_t \times n_t}. \quad (23)$$

The fully weighted projection would replace (21) by

$$(\tilde{\rho}, b)^{k+1} = \arg \min_{(\tilde{\rho}, b) \in \mathcal{C}_h^\varepsilon(\Delta t)} \sum_j \Delta t_j^{\text{dual}} \Delta x (\tilde{\rho}_{j,i} - \mu_{j,i})^2 + \sum_{n,i} \Delta t_n \Delta x (b_{n,i} - \beta_{n,i})^2. \quad (24)$$

As with the y -update, Δx is a single global constant shared by every term here (space is never graded in this note), so it factors out and cancels from the argmin exactly like any shared constant weight – the only physically meaningful content of (24) is the *time* weight, Δt^{dual} and Δt .

Standard fact for weighted least squares onto a linear constraint $\{Lx = d\}$: minimizing $(x - v)^\top W(x - v)$ for SPD diagonal W , instead of the plain $\|x - v\|_2^2$, replaces every plain adjoint $L^\top$ in the normal equations by the W -adjoint $L^{*W} := W^{-1}L^\top$,

$$x^{k+1} = v - W^{-1}L^\top(LW^{-1}L^\top)^{-1}(Lv - d) \quad (25)$$

– i.e. (10)/(9) with every $D_t^\top, A_t^\top$ replaced by D_t^{*W}, A_t^{*W} . Concretely for D_t : the vector $D_t^\top$ acts on lives in the *cell*-indexed space (Section 6.1), whose natural physical weight is $\Delta x \text{diag}(\Delta t)$; using $(D_t^\top v)_j = v_{j+1}/\Delta t_{j+1} - v_j/\Delta t_j$,

$$D_t^{*W}(v)_j := \frac{1}{\Delta t_j^{\text{dual}}} (D_t^\top [\Delta t \odot v])_j = \frac{1}{\Delta t_j^{\text{dual}}} \left(\Delta t_{j+1} \frac{v_{j+1}}{\Delta t_{j+1}} - \Delta t_j \frac{v_j}{\Delta t_j} \right) = \frac{v_{j+1} - v_j}{\Delta t_j^{\text{dual}}}, \quad (26)$$

which collapses to the ordinary divided difference of v on the dual/staggered grid – the textbook “physically natural” finite-difference adjoint. Its kernel is exactly $\text{span}\{1_{n_t}\}$: $D_t^{*W}(1_{n_t})_j = (1-1)/\Delta t_j^{\text{dual}} = 0$ identically, recovering the continuum’s constant-gauge direction that Remark 3 says the plain Euclidean adjoint fails to reproduce (Proposition 3). So (26) is, explicitly, the substitution Remark 3 refers to.

But swapping $D_t^\top \rightarrow D_t^{*W}$ (and $A_t^\top \rightarrow A_t^{*W}$, analogously) into (10) does more than rescale M^k ’s entries: Proposition 2’s proof is an algebraic identity built on D_t, A_t appearing *together with their own plain transposes* in $(D_t + \varepsilon \lambda_x^k A_t)(D_t + \varepsilon \lambda_x^k A_t)^\top$. Weighting only the transpose side of that product (leaving D_t, A_t themselves unweighted) breaks the cancellation the proof relies on – and empirically, in this codebase, the resulting operator no longer satisfies the projection’s idempotency identity (Remark 3). Re-deriving a correct SPD, idempotent projection under this weighted metric from scratch – if one exists – is exactly the open problem Remark 3 flags; (21), the plain Euclidean projection, is what the reference implementation uses today.

Summary

prox	objective (today)	weight	penalty/metric weight (today)	fully $\Delta t \Delta x$ -weighted
x -update (pro- jection)	n/a (hard straint)	con-	Euclidean, weighted	un- needs D_t^{*W}, A_t^{*W} – breaks idempotency (open)
y -update (KE prox)	relative $\Delta t_n/\Delta t_{\text{ref}}$		Euclidean, weighted	un- cancels completely $\rightarrow$ (17)
diagnostics (<code>norm_fn</code>)	n/a		genuine $\Delta t_n \Delta x$	already exactly this

12 What is gained, what is lost

12.1 Gained

Local time resolution exactly where the marginal boundary data forces it, at a cost in cell count that is $O(2^{n_b})$ per end rather than $O(\bar{n}_t)$ globally (grading $n_b = 5$ cells costs 62 extra

cells per end; matching that same near-boundary resolution with a *uniform* grid would require shrinking Δt everywhere, i.e. $O(\bar{n}_t \cdot 2^{n_b})$ total cells). Empirically (`scripts/sb.gaussian_refine_ends_vs_uniform.py`), at matched *base* resolution $\bar{n}_t$, grading the first/last 3 cells cut the mean L^2 error against the Sinkhorn/analytical reference by roughly $3\times$ over the plain uniform grid at the same $\bar{n}_t$, concentrated almost entirely in the first/last $\sim 10\%$ of the time domain (the interior, unrefined portion shows no measurable difference, as expected from Section 6.1: every operator away from the graded region is untouched).

12.2 Lost – precisely one step: the FP-projection’s DC (spatial $k=1$) mode

Everything else in the LADMM iteration is exactly as expensive as on a uniform grid of the same total n_t :

- Prox_{J_h} (the KE prox): pointwise in (τ_j, ξ_i) , never touches Δt_j except as a per-row *scalar* multiplier on the step size (Section 10: $\sigma_{\text{eff}} = \sigma \Delta t_j / \Delta t_{\text{ref}} - O(n_t n_x)$, unaffected).
- The consensus map $\mathcal{A}$ and its adjoint: by Lemma 1, A_t is completely unchanged; the only Kronecker-factor that changes (D_t) never appears in $\mathcal{A}$ at all (Eq. (16) only uses A_t, A_x) – $O(n_t n_x)$, unaffected.
- The $n_x - 1$ non-DC tridiagonal solves ($k = 2, \dots, n_x$): still $O(n_t)$ each via a row-varying (rather than constant) Thomas sweep, still fully vectorizable across modes – $O(n_t n_x)$ total, unaffected, and Theorem 1 shows this remains as numerically safe as the uniform-grid case, unconditionally in $\{\Delta t_j\}$.

The one place cost genuinely increases is the single $k=1$ (DC) spatial mode’s solve (Section 7): $O(n_t \log n_t)$ (spectral, via T_t) $\rightarrow O(n_t^2)$ (dense pseudo-inverse mat-vec), *every LADMM iteration*, plus a one-time $O(n_t^3)$ factorization cost at precompute (vs. $O(n_t)$ to just read off the closed-form eigenvalues λ_t^i). Since this is only 1 of n_x spatial modes, it is asymptotically dominated by the other $n_x - 1$ modes’ $O(n_t n_x)$ total whenever $n_t \lesssim n_x \log n_x$ – but for a *time-refined* grid specifically (large n_t , modest n_x , exactly the regime grading targets), the $O(n_t^2)$ DC-mode term can come to dominate the entire projection cost. A specialized $O(n_t)$ – $O(n_t \log n_t)$ solver for this one tridiagonal-with-known-rank-deficiency system (exploiting Proposition 3’s explicit null vector, rather than a generic dense pseudo-inverse) is the concrete, actionable optimization this note leaves open.

12.3 ETD: generalized, but its expected advantage did not clearly show up

Section 9 generalizes the ETD scheme to non-uniform Δt with the same closed-form character as the CN generalization – no efficiency regression beyond the one already identified (the DC mode is literally the same singular system either way). The motivation for doing so was the main paper’s Fig. 6 finding that CN rings once $\varepsilon / \Delta t \gtrsim 1$, exactly the regime grading pushes toward locally. The head-to-head test in Section 9’s remark did not confirm a clear advantage at the settings tried – both schemes behaved comparably. So: ETD-nu is available and validated (`projection_expsemi_nu.py`, `validate_nu.py` checks 6–8) as a drop-in alternative to CN-nu wherever the main paper’s own reasoning would recommend it, but this note does not claim to have empirically demonstrated the stability gain that motivated building it – a broader sweep (more aggressive grading, larger ε , non-Gaussian marginals) is needed before drawing a real conclusion either way.

13 Conclusion

The LADMM scheme, its consensus map, and its KE prox generalize to a dyadically graded non-uniform time grid with no algorithmic change and no loss of the SPD/stability guarantee that makes the uniform-grid tridiagonal solve safe (Theorem 1, proved with the exact same argument as Lemma A.2). Both the CN and ETD schemes generalize this way, the latter with no probing needed anywhere (Section 9) – both are implemented and validated to machine precision against their uniform-grid counterparts. The entire efficiency cost of non-uniformity is isolated to a single, identifiable step, shared by both schemes – the singular DC spatial mode’s linear solve inside the Fokker–Planck projection – which degrades from a closed-form $O(n_t \log n_t)$ spectral divide to a dense $O(n_t^2)$ solve, because the discrete time-Laplacian is no longer diagonalized by a fixed fast transform once Δt is non-uniform. A specialized solver for that one mode remains the concrete, actionable optimization this note leaves open; a broader empirical sweep to actually pin down when (if ever) ETD-nu earns its keep over CN-nu on a graded grid is the other.